# microGPT – Detailed Mathematical Reference

## Token Embedding

Tokens are converted into vectors via an embedding matrix E.
e_i = E[token_i]

**Interpretation:** Each token selects one row from E.
No multiplication needed — this is a lookup operation.

## Positional Encoding

Since Transformers lack recurrence, positional information is added:
x_i = e_i + p_i

where p_i is the positional embedding.
This preserves dimensionality while injecting order.

## Scaled Dot■Product Attention (Derivation)

Step 1: Compute similarity scores
score = QK■

Step 2: Scale to prevent large gradients
score_scaled = (QK■) / √d_k

Step 3: Normalize via softmax
A = softmax(score_scaled)

Step 4: Weighted sum of values
Attention = A V

## Softmax Function

Converts logits into probabilities:
softmax(z_i) = exp(z_i) / Σ_j exp(z_j)

Ensures outputs sum to 1.

## Layer Normalization (Derivation)

μ = mean(x)
σ² = variance(x)

LN(x) = γ ((x – μ) / √(σ² + ε)) + β

Normalizes per token, stabilizing training.

## Feed■Forward Network

FFN(x) = max(0, xW■ + b■) W■ + b■

Introduces non■linearity and feature mixing.

## Residual Connections

y = x + F(x)

Helps gradient flow in deep networks.

## Cross■Entropy Loss

L = – Σ y_i log(p_i)

For correct class:
L = – log(p_correct)

Penalizes confident wrong predictions.

## Gradient Descent

θ ← θ – η ∇L(θ)

Updates parameters to minimize loss.

## Conceptual Diagram – Attention Flow